Deployment of Cryptography

Lecture #03

States of Data

In a modern information system, data can be in various states

- Data at rest
  - This refers to data that is stored on various storage devices
- Data in motion
  - This refers to data while it is being communicated over any sort of communication channel
- Data in use
  - This refers to data when it is the main memory of a computer and potentially being processed by a processor

Protecting Data at Rest

Deployment of cryptography

Options to deploy cryptography to protect data at rest

- Application layer
  - The applications are built/modified to explicitly use cryptographic functions
- Infrastructure layer
  - The file/disk storage system incorporates encryption
- Each of these options involves some tradeoffs

Application Layer Encryption

- The usual method for an application interacting with a database using a database language such as SQL is not by literal SQL statements but by “prepared” SQL statements that are used from within an application.
- The application then explicitly manages encryption/decryption

Example- Using SQL from a programming language – an example using python

pip3 install psycopg2 --- install the interface package

```
import psycopg2
conn = psycopg2.connect(database="db_name",
                        host="db_host",
                        user="db_user",
                        password="db_pass",
                        port="db_port")

cursor = conn.cursor()
cursor.execute("SELECT * FROM DB_table WHERE id = 1")
```

To be written in the python program

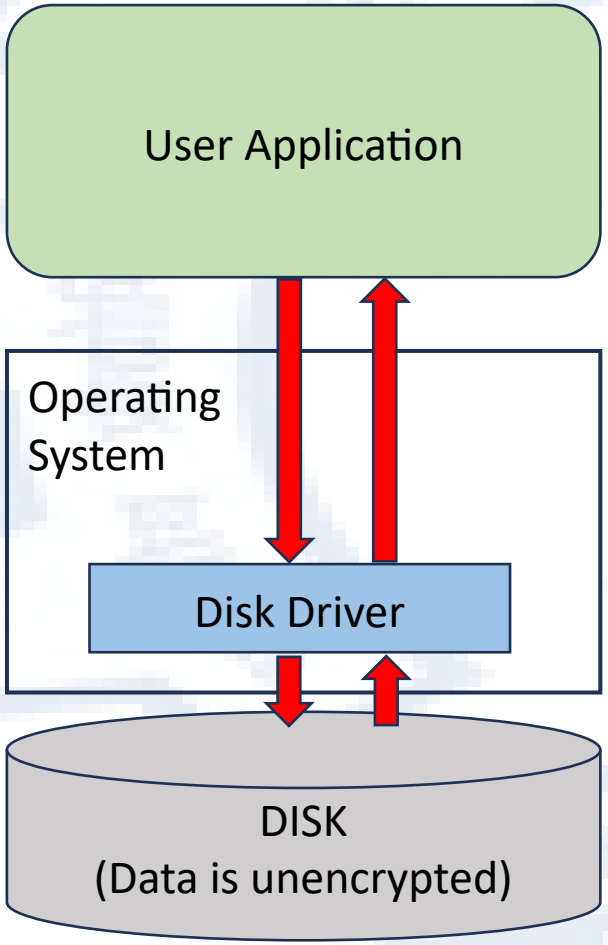
"prepared" SQL statement

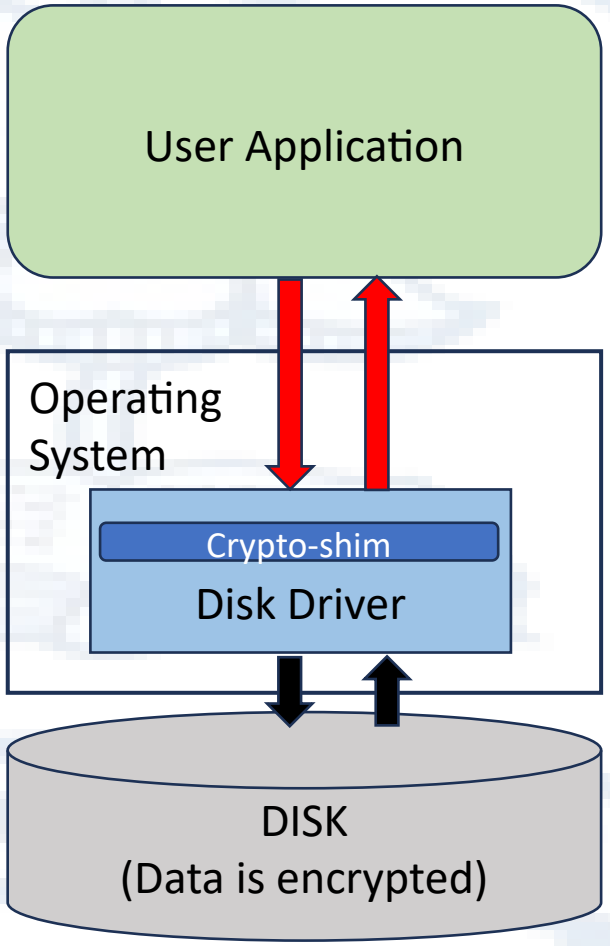
Implementing encryption

```
cursor = conn.cursor()
……..
# load values into variables here
name= ….
phno= ….
key= ….
resdate= ….
restime= ….
diners= ….
insstr="INSERT INTO Restaurant_Table_Reservations (" + name + "," , pgp_sym_encrypt(phno ,
            key) + "," + resdate + "," + restime + "," + diners + ");"
cursor.execute(insstr)
```

The important question is how do we keep "key" secure?

Transparent Disk Encryption (TDE)

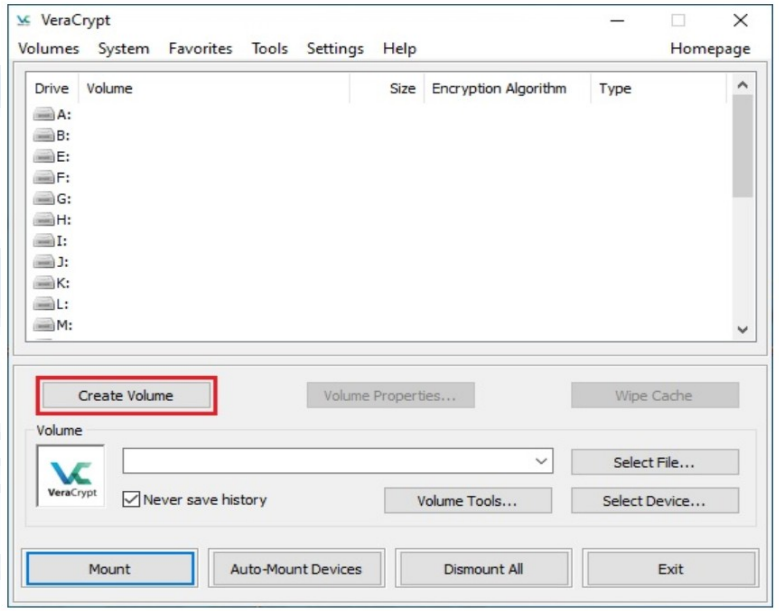


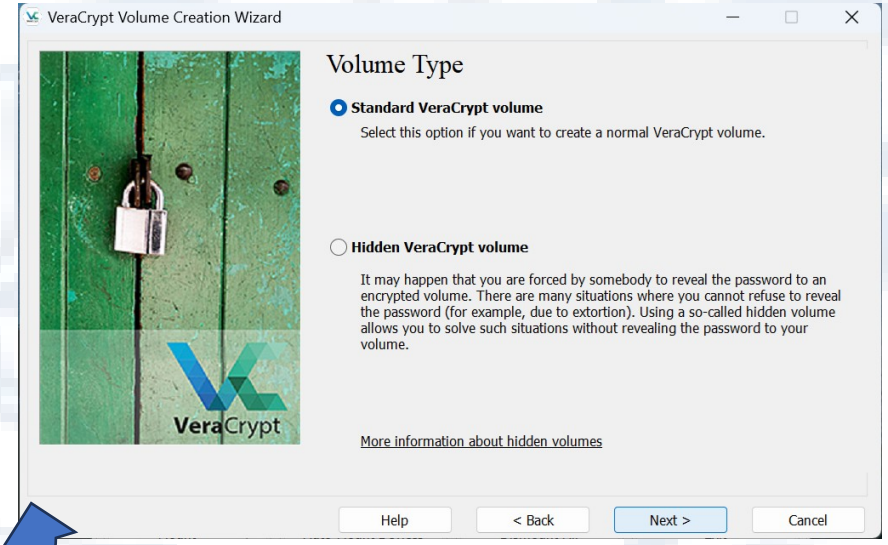


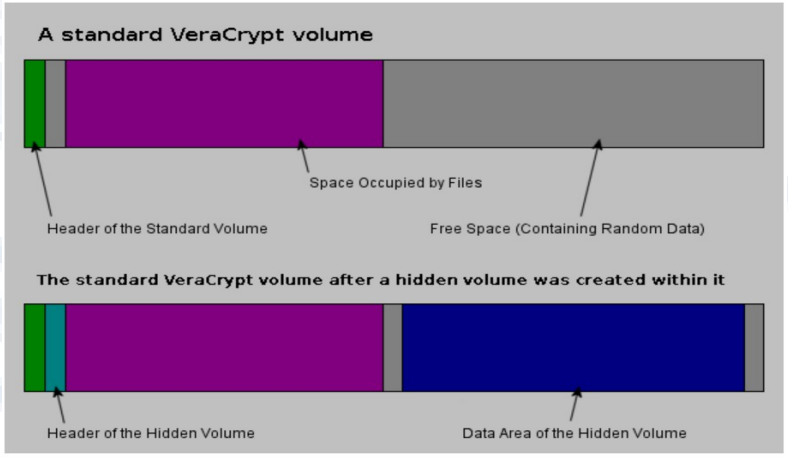
- The crypto-shim intercepts all data going to and from the disk
- All data going to the disk is encrypted
- All data coming from the disk is decrypted
- The crypto-shim must be provided with a key which will be used to do the encryption/decryption
- Applications are not affected in any way
- This type of scheme can be designed to work at a folder or file system level as well

An Example- Veracrypt

- A flexible transparent disk encryption product
- Open source
- Free for personal use
- Allows plausible deniability







TDE – Benefits and Drawbacks

- Benefits
  - **No changes are needed to the applications**
  - Several tested proprietary and open source implementations exist
  - Most operating systems have native capabilities for disk encryption
    - Bitlocker for Windows and LUKS for Linux
    - Third party products exist that can do transparent disk and filesystem encryption
- Drawbacks
  - All data is encrypted and this may create a performance impact
  - With this sort of scheme data is accessible on an all or nothing basis. A person that has access to any data on the encrypted storage has access to all the data on that storage

Transparent Database encryption

- Transparent Database encryption is conceptually similar to installing the database on a disk with transparent disk encryption.
  - The advantages of Transparent Database encryption are the same as transparent Disk Encryption
  - The disadvantages of Transparent Database encryption are the same as transparent Disk Encryption
  - Transparent Database Encryption can be used when it is not considered desirable or practical to encrypt the entire disk and the database system being used supports transparent database encryption
- Not all database management systems support Transparent Database encryption

Major Databases that support Transparent Database Encryption

- Postgres
  - The community versions do not provide Transparent Database Encryption but the EDB Postgres Extended Server from version 15 and EDB Postgres Advanced Server
- MySQL
  - MySQL provides a Keyring feature that allows various key management solutions to be plugged in to enable its Transparent Database encryption function
- Oracle
  - Oracle provides Transparent Database Encryption Functionality
- Microsoft SQL Server
  - SQL Server provides Transparent Database Encryption Functionality

Protecting Data in Motion

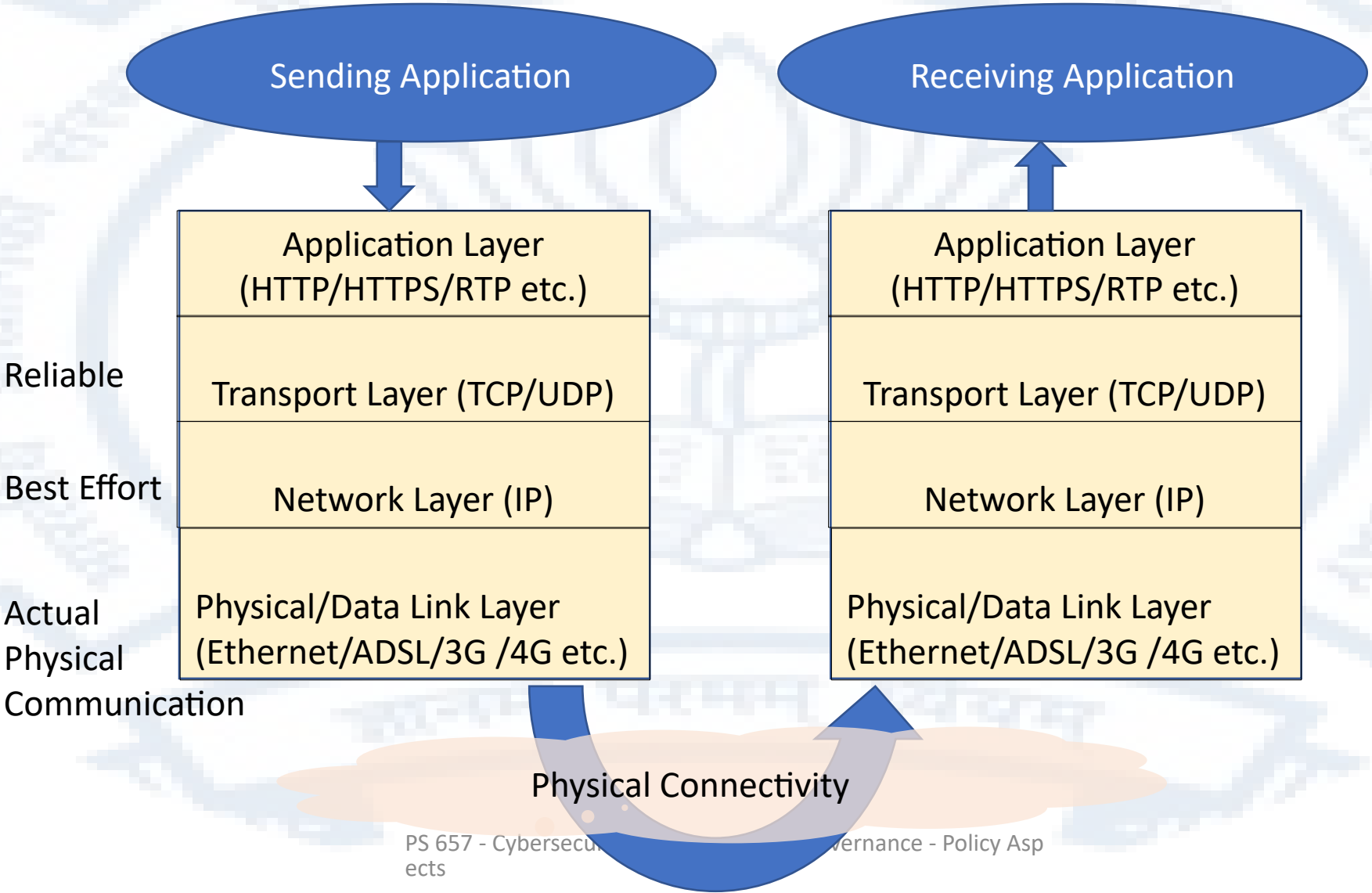
Deployment of cryptography

Options to protect data in motion

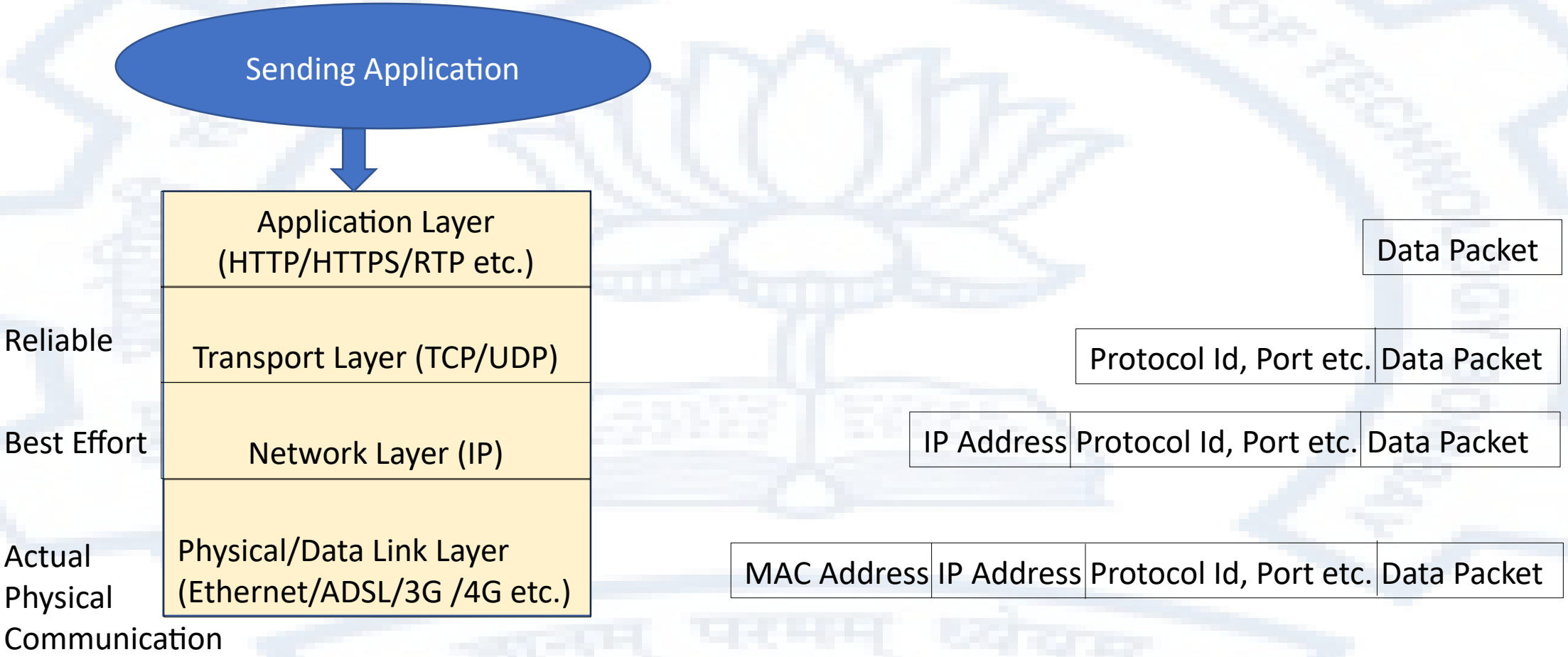
- At the physical layer – Link encryptors
- At the data-link layer – MACSEC
- At the network layer – IPsec
- At the transport layer – TLS
- At higher layers – HTTPS

- Each option has its pros and cons but the most common ways of deployment are TLS and HTTPS followed by site-to-site IPsec (VPNs)

Concept of layered communication

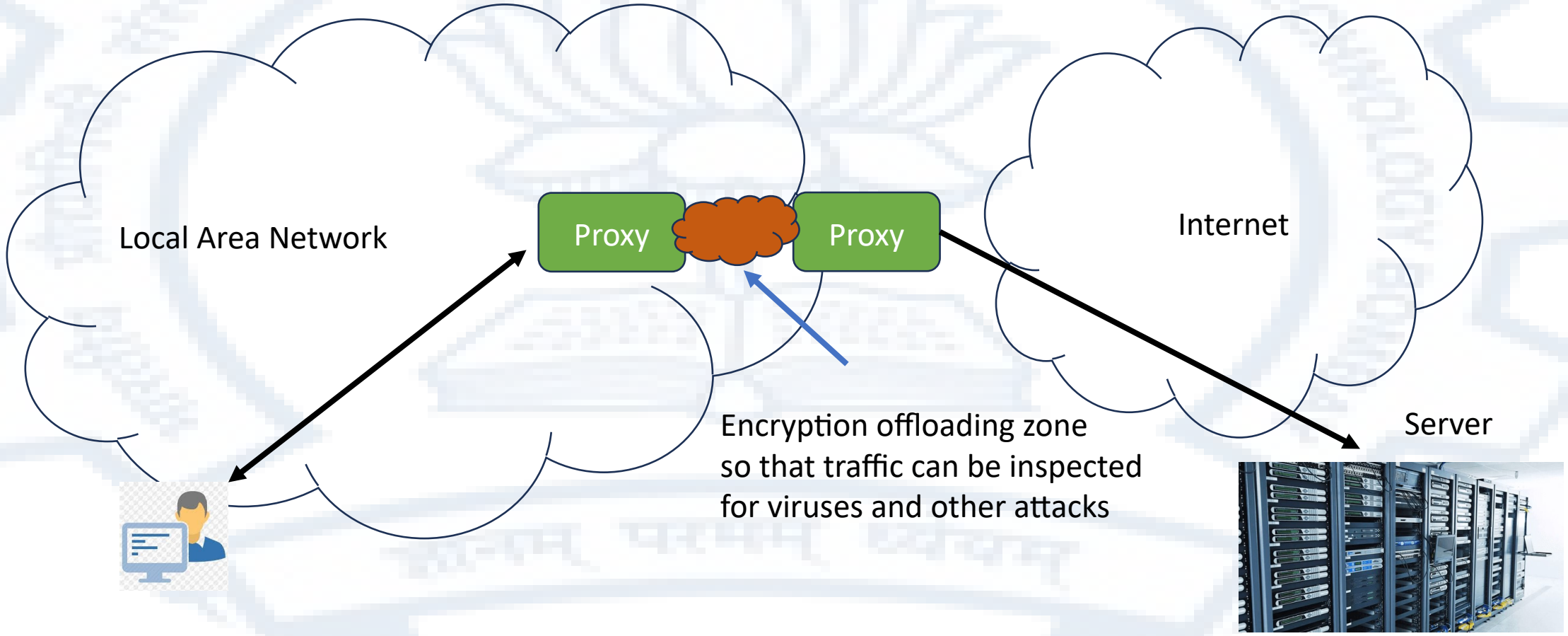


Progressive encapsulation of data



The deployment options are to chose which level to encrypt/decrypt at

Practical aspects – Encryption creates challenges for security inspection systems



Protecting Data in Use

Deployment of cryptography

Methods of protecting data in use

- Hardware supported OS mechanisms that protect data in main memory
  - Protection rings in operating systems
  - Special registers for memory protection
  - Dedicated single user modules isolate computation
- Computational techniques such as multiparty computation
- Homomorphic encryption – the holy grail

Privacy Preserving Computation
Secure Multiparty Computation

Secure multi-party computation (also known as **secure computation**, **multi-party computation** (**MPC**) or **privacy-preserving computation**) is a subfield of cryptography with the goal of creating methods for parties to jointly compute a function over their inputs while keeping those inputs private. Unlike traditional cryptographic tasks, where cryptography assures security and integrity of communication or storage and the adversary is outside the system of participants (an eavesdropper on the sender and receiver), the cryptography in this model protects participants' privacy from each other.–

https://en.wikipedia.org/wiki/Secure_multi-party_computation

Secure Multiparty Computation

- Three people have salary A, B and C respectively
- They want to know the total of their salaries without disclosing their Individual salaries to anyone
- Each generates a random number x, y and z respectively
- Each now sends the random number to one of the other participants and their salary minus the random number to the other
- Each participant adds the numbers received from the others to their own salary and sends the sum to both other participants
- Each participant adds all the numbers it has received and the total works out to be the sum of the salaries of all.

Privacy Preserving Computation Homomorphic Encryption

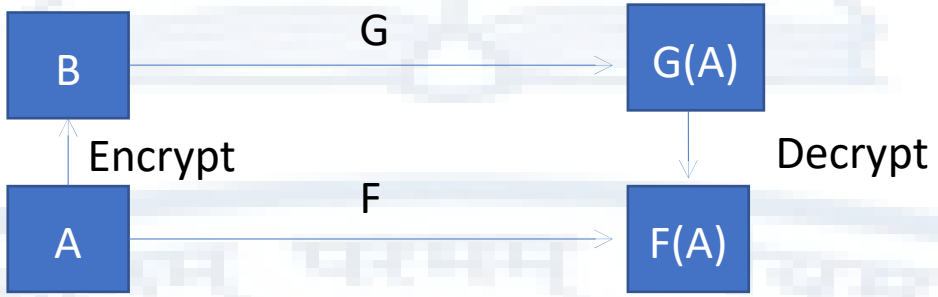
Homomorphic encryption *is a form of encryption that permits users to perform computations on its encrypted data without first decrypting it. These resulting computations are left in an encrypted form which, when decrypted, result in an identical output to that produced had the operations been performed on the unencrypted data. Homomorphic encryption can be used for privacy-preserving outsourced storage and computation. This allows data to be encrypted and out-sourced to commercial cloud environments for processing, all while encrypted. –*

https://en.wikipedia.org/wiki/Homomorphic_encryption

Homomorphic Encryption

- The holy grail of encryption
- Shown to be theoretically possible

Craig Gentry, Fully Homomorphic Encryption Using Ideal Lattices, In *the 41st ACM Symposium on Theory of Computing (STOC)*, 2009



Key Management

Deployment of cryptography

Comparison of symmetric key and public (asymmetric) key systems

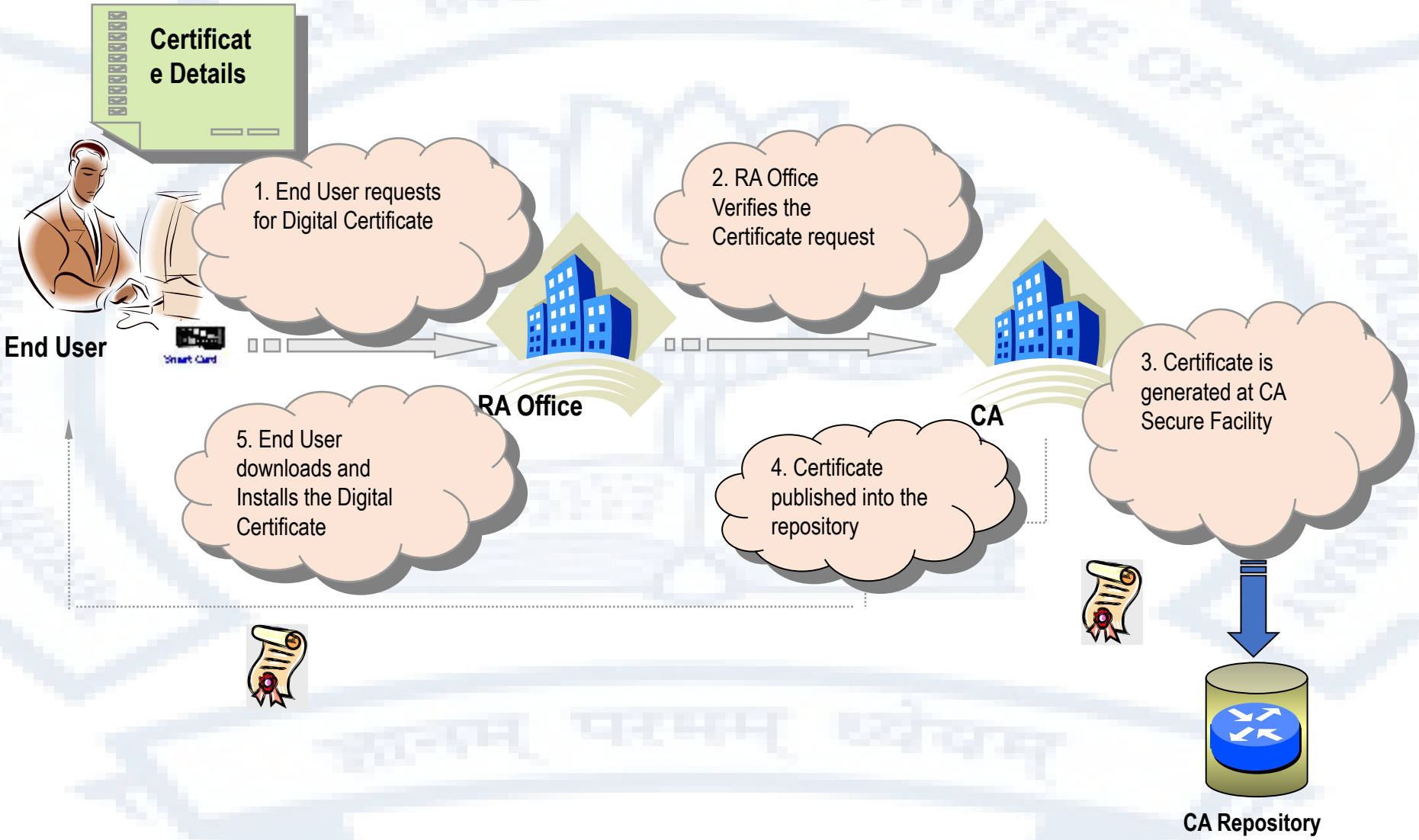
Aspects	Symmetric Systems	Public Key Systems
Uses	Confidentiality	Confidentiality, Digital Signature
Speed	Fast	Relatively slow
Keys needed for *n* participants (n=3000)	${}^{n}C_{2}$ (44,98,500) 2999 with each person	***2n*** (6000) 1 with each person 3000 in repository
Recommended Use	Confidentiality	Key management, Digital Signature

Therefore in practice it is necessary to use **hybrid** systems .

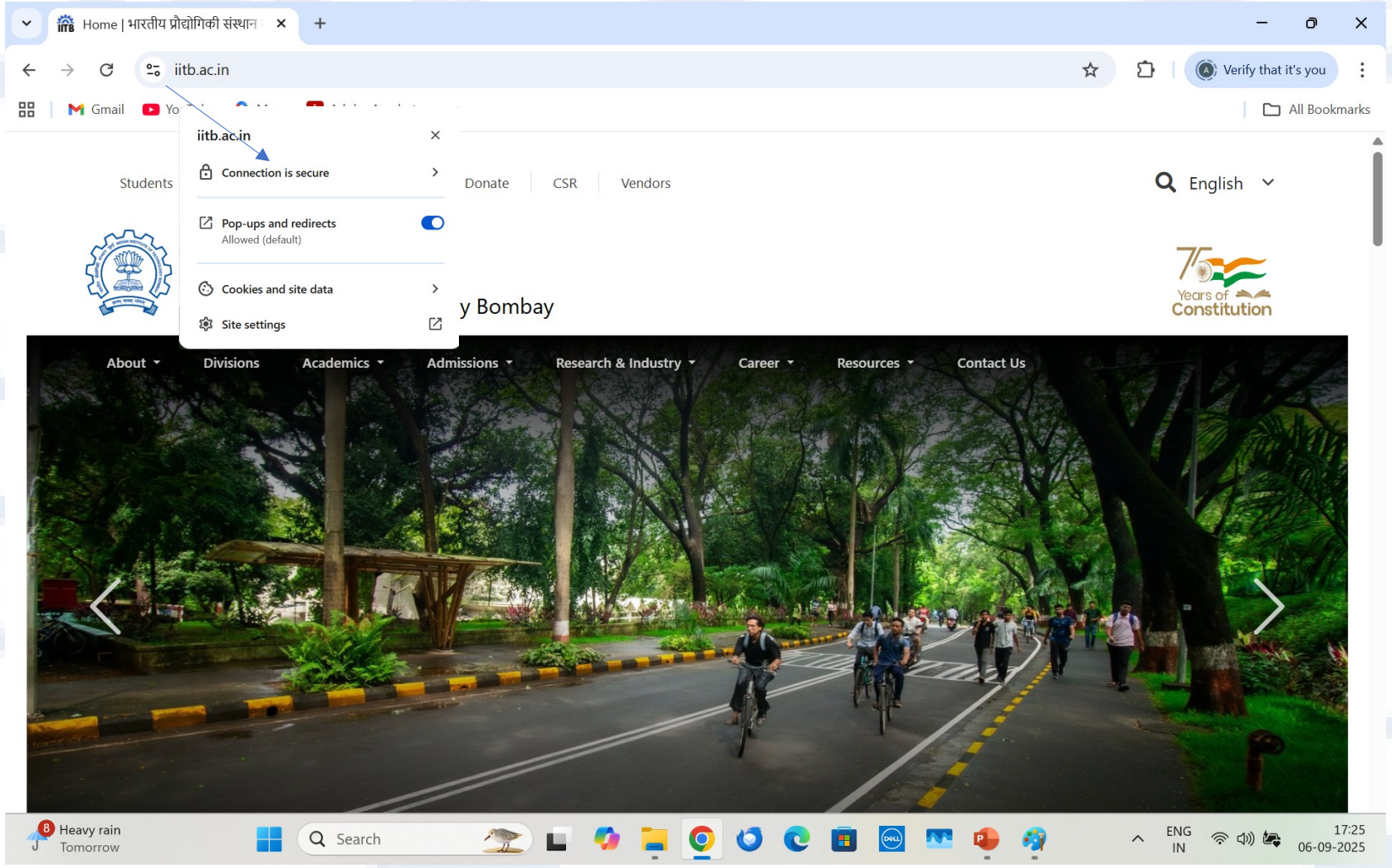
Public Key Infrastructure

- The Diffie-Hellman system is unauthenticated. It can be used to negotiate a shared secret between two parties but neither is sure of the identity of the other
- In the RSA system, when the public key of a recipient for a communication is requested, again there is no surety that the public key really belongs to the entity that it is claimed to be the public key of
- This problem is solved by the deployment of Public Key Infrastructure

Typical PKI deployment



Digital Certificates



To summarize

- Data needs to be secure whenever it is at rest, in motion or in use
- There are several deployment options that are available in each case and each of those options has its own pros and cons
- Practical deployments of cryptography employ both symmetric and public keys systems for efficiency
- Public key cryptography provides an important part of the solution for key management
- Key management additionally needs several processes and secure key containers to be effective

End of

Lecture #03